

1 Software Quality: Design, Astrazione e Processo

La prima parte del corso introduce i principi fondamentali per sviluppare software di qualità, mettendo al centro il ruolo della progettazione, dell’astrazione e dell’organizzazione del processo di sviluppo. In un contesto in cui il software rappresenta una componente sempre più rilevante dell’economia globale e incide in modo significativo sul PIL mondiale¹ produrlo in modo efficace e sostenibile è una sfida cruciale: non basta far “funzionare” un sistema, occorre progettare affinché sia manutenibile, evolvibile e coerente con le esigenze aziendali. Il corso si propone quindi di analizzare i problemi tipici dello sviluppo software e di presentare metodologie moderne che consentano di affrontarli in modo strutturato, evidenziando come buone pratiche, principi di astrazione e processi adeguati non solo migliorino la qualità del prodotto finale, ma rendano anche il lavoro di ingegneri e sviluppatori più solido, consapevole e sostenibile nel tempo.

1.1 Necessità del processo di sviluppo

La produzione di software di qualità è un’attività intrinsecamente complessa. Se da un lato è relativamente semplice realizzare piccoli prototipi funzionanti in tempi rapidi, dall’altro questi frammenti, sviluppati senza una visione strutturata, rischiano di risultare incoerenti, difficili da integrare e poco riutilizzabili. Il passaggio da “qualcosa che funziona” a un sistema solido, manutenibile e pronto per la produzione richiede infatti uno sforzo molto maggiore di quanto spesso si immagina. Lo sviluppo software è inoltre fortemente esposto a errori, talvolta sottili e difficili da individuare, soprattutto quando il codice è frammentato, poco leggibile o frutto di scelte progettuali improvvisate.

Non bisogna poi dimenticare che sviluppare software non significa interagire solo con la tecnologia, ma anche con persone, team e organizzazioni. Requisiti instabili, stime eccessivamente ottimistiche e pratiche poco ingegnerizzate possono compromettere l’evoluzione lineare di un progetto, generando ritardi, frustrazione e, nei casi peggiori, il fallimento completo.

¹si stima che lo sviluppo software rappresenti il 15% del PIL globale e che raggiungerà il 30% nel 2030

Se lo sviluppo è così complesso, diventa fondamentale imparare dall'esperienza e chiedersi quale sia la “qualità” che rende un progetto realmente di successo. Quali pratiche di processo e quali tecniche di programmazione favoriscono la produzione di software robusto ed evolvibile? In un contesto dinamico, il software deve sapersi adattare all'instabilità dei requisiti, contenere i costi di manutenzione e supportare efficacemente la prevenzione e la rimozione degli errori.

Produrre buon software significa comprendere le caratteristiche che hanno reso efficaci altri sistemi, adottare principi e pattern consolidati e organizzare l'intero processo di sviluppo in modo consapevole e strutturato. Questo è, in sintesi, l'obiettivo del corso: fornire strumenti tecnici e organizzativi per costruire software di alta qualità. Tra le competenze chiave necessarie, una emerge in modo particolare: la capacità di comprendere e gestire le *astrazioni*.

Per astrazione si intende, in generale, il processo attraverso cui si riconoscono caratteristiche comuni tra più entità concrete, isolandole dai dettagli specifici per formare un concetto più generale. Da un punto di vista filosofico, l'astrazione è il meccanismo cognitivo con cui individuiamo proprietà condivise tra oggetti o fenomeni diversi e costruiamo un'idea unitaria che li rappresenti. È il passaggio dal particolare al generale.

Nel contesto dello sviluppo software, l'astrazione diventa uno strumento operativo per governare la complessità. Significa modellare un sistema mettendo in evidenza gli aspetti rilevanti per la progettazione. In pratica, il codice dovrebbe esprimere direttamente l'idea che rappresenta, senza essere appesantito da particolari superflui. I classici esempi concreti sono: evitare la duplicazione (principio DRY) e far dipendere i moduli da interfacce e concetti astratti piuttosto che da dettagli concreti (principio DIP).

1.2 Qualità del Software: criteri di definizione

Non esiste una definizione univoca di qualità del software, perché essa dipende dal punto di vista adottato.

- Da una prospettiva più filosofica, la qualità richiama concetti come eleganza, coerenza e “bellezza” del codice: un sistema è di qualità quando è pulito, leggibile e costruito con cura, tanto da risultare comprensibile anche a chi non lo ha scritto.
- Dal punto di vista del prodotto, invece, la qualità è legata all'aderenza a standard e a caratteristiche misurabili, come affidabilità, prestazioni o sicurezza.

- Nella prospettiva del committente, un software è di qualità se soddisfa pienamente i requisiti concordati.
- Per l'utente finale, infine, la qualità si traduce in un'esperienza efficace, intuitiva e piacevole.

Accanto a queste prospettive, è utile distinguere tra qualità esterna e qualità interna. La qualità esterna riguarda il comportamento osservabile del sistema: correttezza, efficienza, usabilità, robustezza. È ciò che utenti e stakeholder percepiscono direttamente. La qualità interna, invece, riguarda la struttura del software e la sua organizzazione: modularità, leggibilità, testabilità, flessibilità ed evolvibilità. È ciò che determina quanto sia semplice estendere, modificare o mantenere il sistema nel tempo.

Spesso l'attenzione si concentra quasi esclusivamente sulla qualità esterna, perché è quella immediatamente visibile. Questo può generare tensioni: un software può apparire ben funzionante all'esterno ma poggiare su fondamenta fragili. Trascurare la qualità interna per ottenere risultati rapidi porta frequentemente a costi di manutenzione elevati, rigidità e accumulo di problemi strutturali. Per questa ragione, le metodologie moderne promuovono pratiche che rendano le caratteristiche interne più osservabili e verificabili fin dalle prime fasi dello sviluppo.

Qualità del codice ≠ qualità del software

La qualità del software è il risultato complessivo di molti fattori: codice, architettura, dati, librerie, strumenti, piattaforme e processo di sviluppo. La qualità del codice è una condizione necessaria, ma non sufficiente: essa si concentra sulla struttura e sull'organizzazione del programma, e costituisce la base indispensabile per costruire una buona qualità interna.